ROBÓTICA INTELIGENTE

Grado en Ingeniería en Inteligencia Artificial

Introducción a las clases prácticas. Curso 2026-2027

UA
UNIVERSITAT D'ALACANT
UNIVERSIDAD DE ALICANTE
Escola Politècnica Superior
Escuela Politécnica Superior

DFESTS

- Introducción
- Robot Operating System (ROS)
- Contenedores Docker

- **Introducción**
- Robot Operating System (ROS)
- Contenedores Docker

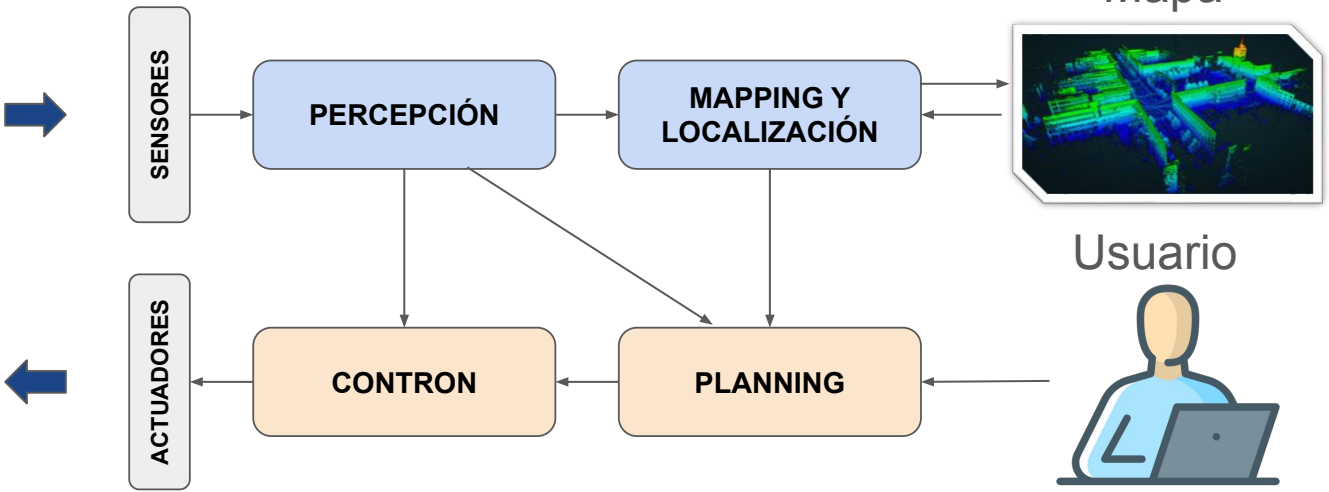
Organización del curso

- El curso se divide en dos prácticas:
  - **Práctica I**: Introducción a las herramientas básicas para robótica móvil y manipulación: ROS2, Gazebo, Navigation Stack, MoveIt y datasets públicos
    - Entrega: 30 de octubre
  - **Práctica II**: Proyecto original
    - Presentaciones: 14-18 de diciembre
    - Entrega: 23 de diciembre
- División en equipos
  - Agruparse en equipos de 4 alumn@s: comunicar al profesor durante las primeras semanas
  - Los equipos serán fijos para todo el curso
  - Solamente se realizará una entrega por equipo

Esquema genérico en robótica

- Un pipeline en robótica es extremadamente modular y asíncrono
  - Por ello se utilizan un entorno específicos para robótica (ROS+Docker)

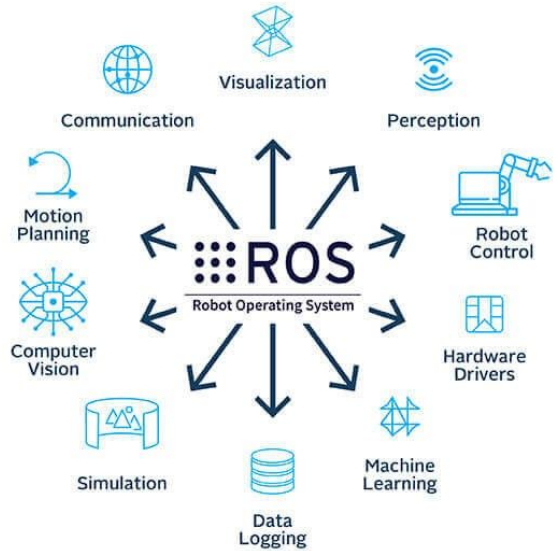




Herramientas que se utilizarán

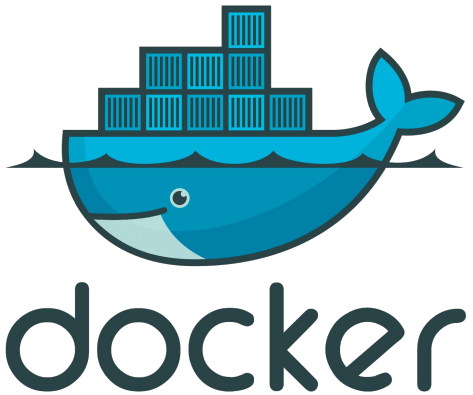
Robot Operating System (ROS)

https://docs.ros.org/en/jazzy/index.html



Docker para la virtualización

https://www.docker.com/



- Introducción
- **Robot Operating System (ROS)**
- Contenedores Docker

¿Qué es (y qué NO es) ROS?

- ¿Qué NO es ROS?
  - No es un Sistema Operativo tradicional (no gestiona el hardware directamente ni reemplaza a Linux o Windows).
  - No es un lenguaje de programación único.
  - No es una aplicación monolítica.
- ¿Qué SÍ es ROS?
  - Es un **Middleware**: una capa de abstracción sobre el sistema operativo.
  - Un marco de trabajo (*framework*) de código abierto.
  - Un conjunto de herramientas, librerías y algoritmos para simplificar el desarrollo robótico.

Filosofía: Reutilización de software modular a nivel global.

ROS 1 vs ROS 2

- ROS 1 (2007) revolucionó la investigación, pero no estaba diseñado para producción ni tiempo real.
- ROS 2 rediseña el núcleo desde cero.

Característica	ROS 1	ROS 2
Comunicación	Protocolo propio (TCPROS)	Estándar industrial DDS (*Data Distribution Service*)
Descubrimiento	Servidor Central (`roscore`)	Totalmente distribuido (Sin punto único de fallo)
Tiempo Real	No soportado nativamente	Soporte para controladores *Real-Time*
Entorno de Red	Redes ideales / locales	Redes inalámbricas inestables / Multi-robot
Plataforma	Principalmente Linux	Multiplataforma (Linux, Windows, macOS)

El Grafo de Computación

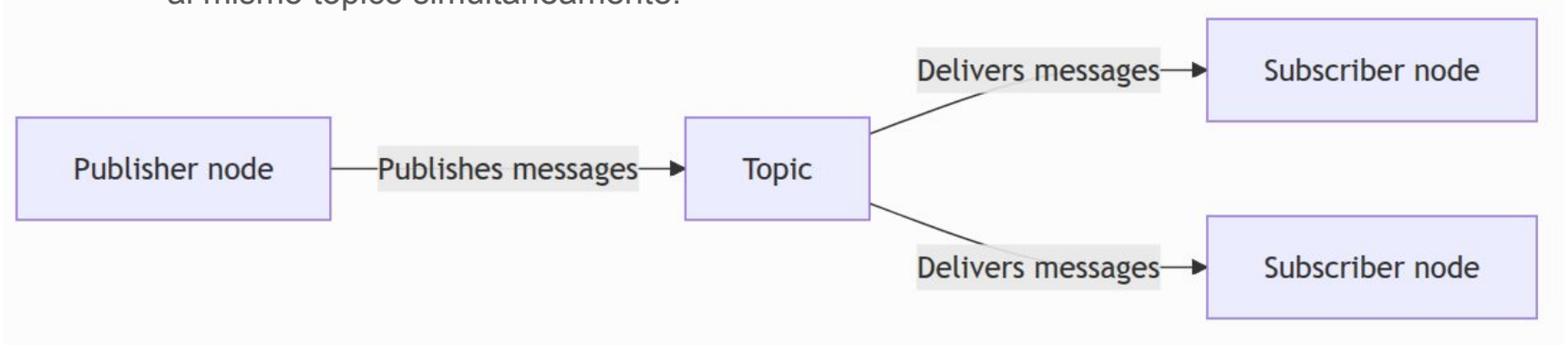
- Un sistema en ROS se organiza como una **red de procesos independientes** que ejecutan cómputos en paralelo.
  - **Nodo (*Node*)**: Entidad básica de procesamiento. Cada nodo realiza una tarea acotada (ej. controlador de motor, filtro de sensor, planificador de rutas).
  - **Desacoplamiento**: Los nodos no necesitan saber qué otros nodos existen; solo se preocupan de la información que envían o reciben.
  - **Ventaja en Ingeniería Robótica**: Si un nodo de procesamiento de imagen falla, el nodo de control de los motores o el de seguridad no colapsan de forma catastrófica.

Comunicación I: Tópicos (*Topics*)

- **Publicador / Suscriptor (*Pub/Sub*).**
  - Comunicación **unidireccional y asíncrona**.
  - Diseñado para flujos continuos de datos (ej. lectura de la IMU a 200 Hz, nubes de puntos LiDAR, odometría).
  - Relación de N x M: Varios nodos pueden publicar o suscribirse al mismo tópico simultáneamente.

Estructura de Mensajes

Tipado estricto (`.msg`).
Ejemplo:
`geometry_msgs/msg/Twist`
(Contiene vectores de velocidad lineal y angular).



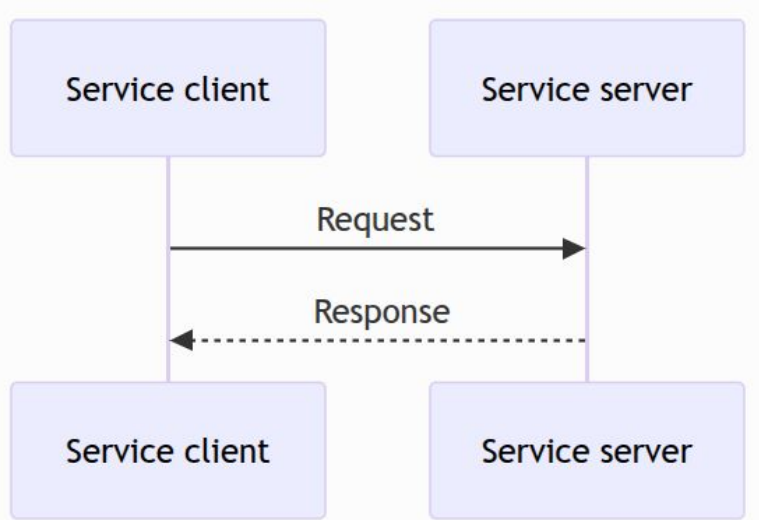
Comunicación II: Servicios (*Services*)

- Paradigma **Petición / Respuesta** (*Request/Response*):
  - Comunicación **bidireccional y síncrona**.
  - Diseñado para operaciones puntuales o rápidas que requieren confirmación explícita (ej. encender un LED, recalibrar un sensor, consultar un mapa).
  - Relación 1 x N: Un cliente envía una petición a un servidor que procesa y devuelve un resultado.

Estructura de Servicio

Definido en archivos `.srv`:

```
Request
---
Response
```



Comunicación III: Acciones (*Actions*)

- Comunicación asíncrona cliente/servidor para **tareas de larga duración**.
- Imprescindibles cuando la tarea puede tardar segundos/minutos y puede requerir **cancelación** o **retroalimentación constante**.
- Ejemplos clave: Desplazarse a un objetivo distante en Nav2, planificar una trayectoria compleja en MoveIt 2.

1. Meta (*Goal*)

El cliente solicita iniciar la tarea (ej. ir a la posición $[X, Y]$).

2. Feedback

El servidor envía datos periódicos (ej. distancia restante al objetivo).

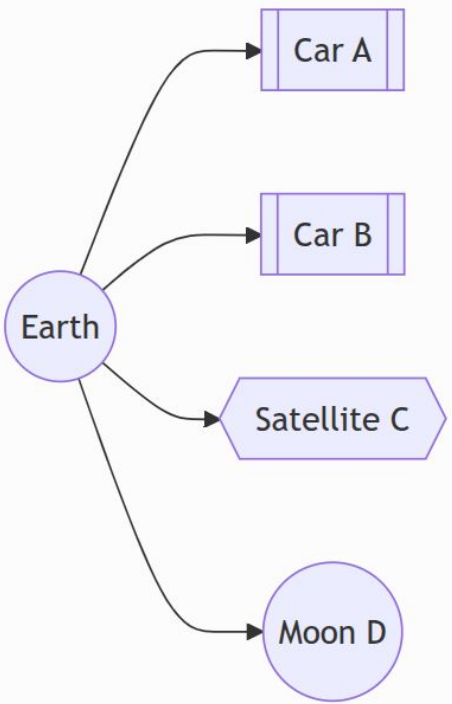
3. Resultado (*Result*)

Confirmación final del éxito o fracaso de la operación.

Sistemas de Referencia y Geometría: *tf2*

Un robot se compone de múltiples marcos de referencia (*frames*) que se mueven entre sí. La librería *tf2* gestiona estas transformaciones tridimensionales continuas en función del tiempo.

- **Jerarquía Estándar de Referencias en Robótica Móvil (REP 105)**:
  - *map*: Marco global del mundo estático (discontinuidades por correcciones del SLAM).
  - *odom*: Marco de odometría acumulada (local, continuo a corto plazo, pero con deriva métrica).
  - *base_link*: Centro rígidamente unido a la base física del robot.
  - *rplidar_link / wrist_3_link*: Marcos de los sensores y efectores finales.



El Ecosistema de Herramientas Auxiliares

ROS 2 no es solo código de comunicación; proporciona una suite de herramientas madura:

- Simulación y Visualización
  - **RViz2**: Herramienta de visualización 3D del estado interno del robot (nubes de puntos, mapas de costes, marcos TF).
  - **Gazebo Harmonic**: Simulador físico 3D para modelar la gravedad, fricción, colisiones y sensores en entornos virtuales.
- Introspección y Experimentación
  - **rqt**: Suite de GUIs dinámicas para graficar tópicos en tiempo real, ver el grafo o inspeccionar parámetros.
  - **Rosbag2**: Grabación y reproducción de flujos de datos reales para experimentación *offline* y pruebas repetibles.

Introducción a la Práctica I

En la **Práctica I** utilizaremos esta arquitectura conceptual sobre cuatro bloques prácticos para inspeccionar y analizar:

1. **Fundamentos ROS**: Dominio de nodos, tópicos, mensajes y herramientas de inspección.
2. **Navegación (Nav2)**: Localización estocástica AMCL y control cinemático sobre un TurtleBot 4.
3. **Odometría LIO**: Procesamiento de datos sensoriales reales y masivos sobre el *Oxford Spires Dataset*.
4. **Manipulación (MoveIt 2)**: Cinemática inversa en el espacio tridimensional con un cobot UR3e.

- Introducción
- Robot Operating System (ROS)
- **Contenedores Docker**

¿Qué es Docker?

Docker: Plataforma de contenedores para aplicaciones.

Aislamiento

Ejecuta aplicaciones en entornos aislados.

Portabilidad

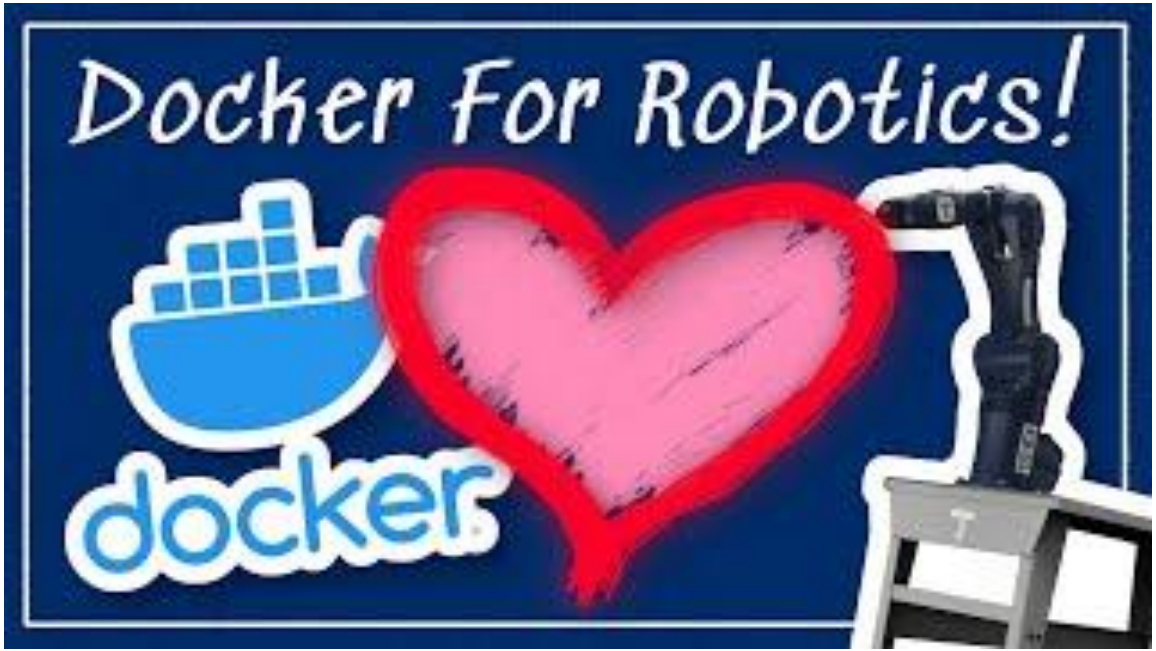
Ejecuta en diferentes sistemas operativos.

Reproducibilidad

Garantiza un entorno consistente.

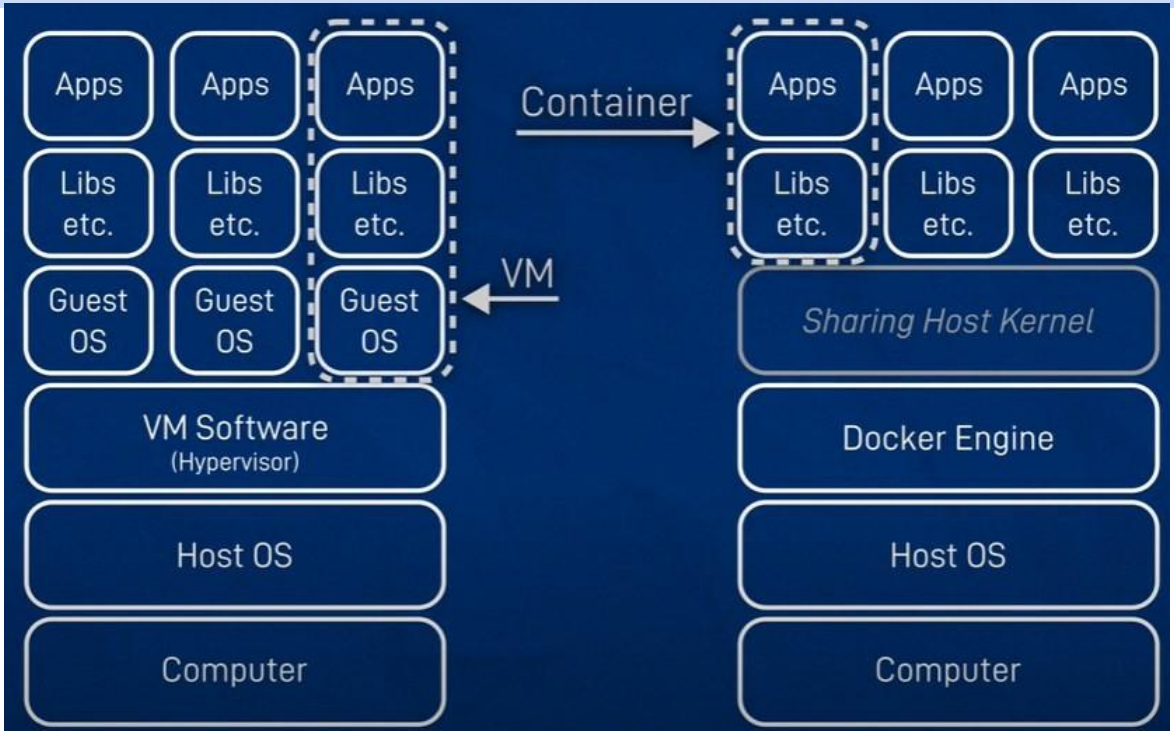


¿Qué es Docker?



https://youtu.be/XcJzOYe3E6M?si=-hH6nA1mwLGzdzdW

Máquina Virtual vs Docker



Beneficios de Docker para ROS

Simplifica el desarrollo y la implementación de robots.

Entorno Consistente

Desarrollos reproducibles.

Gestión de Dependencias

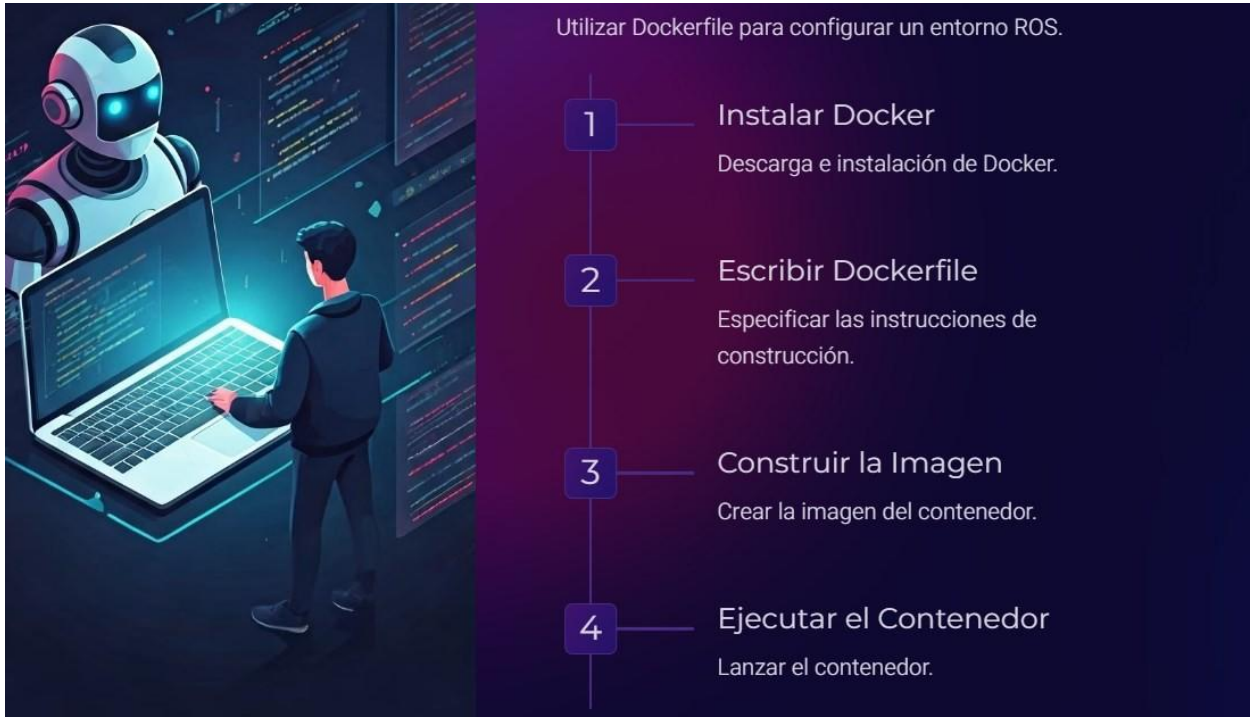
Administración de paquetes de software.

Escalabilidad

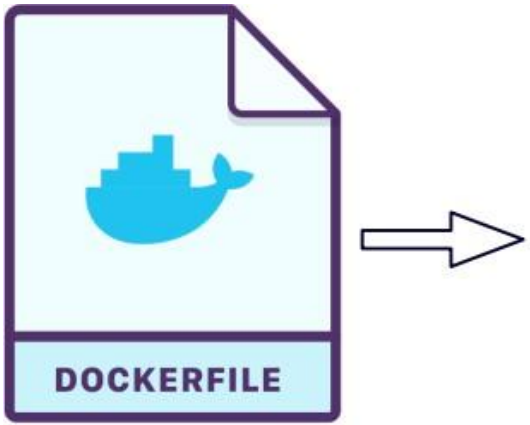
Aumento de recursos según sea necesario.

Creando un contenedor Docker

https://docs.docker.com/engine/install/ubuntu/

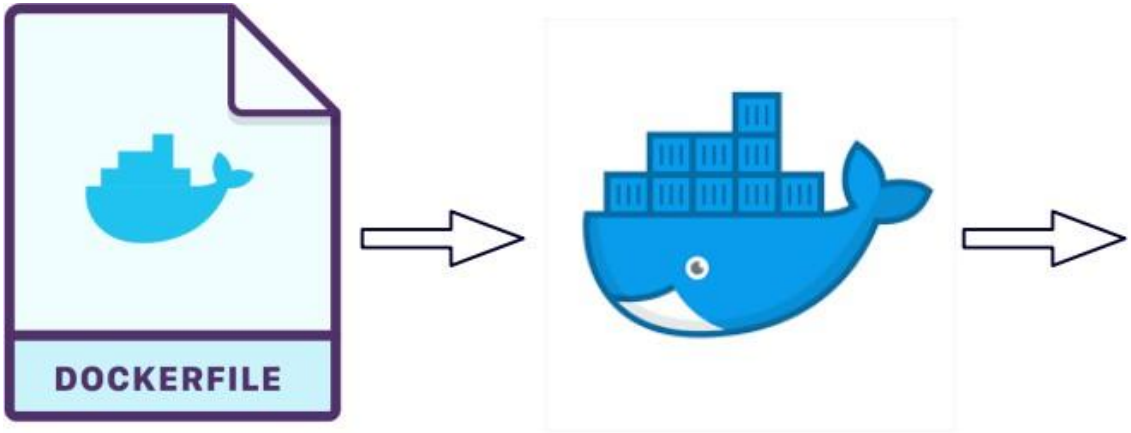


2. Escribir la Dockerfile



Docker file

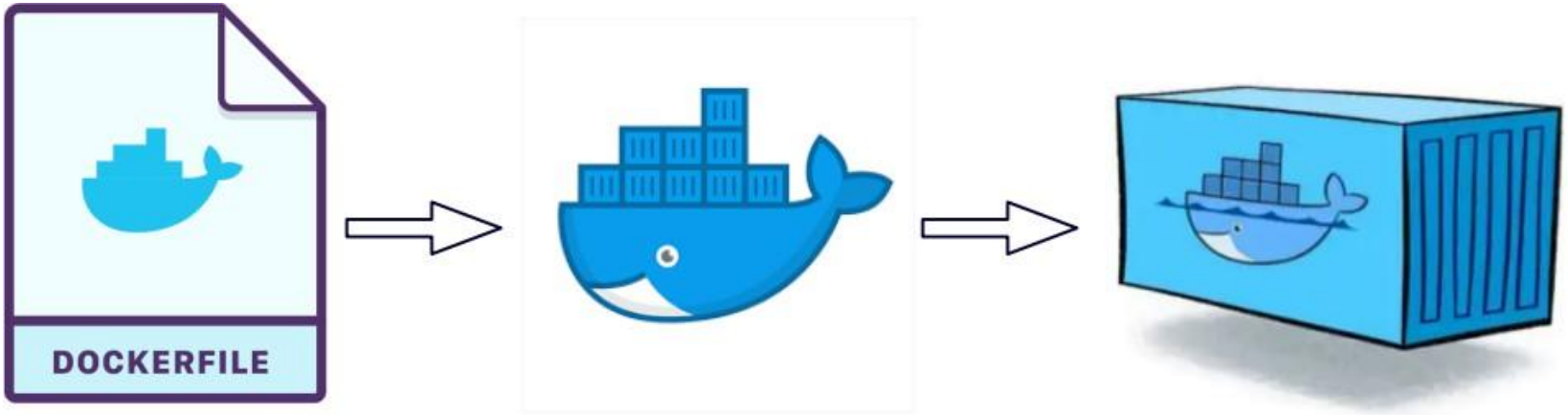
3. Construir la Imagen



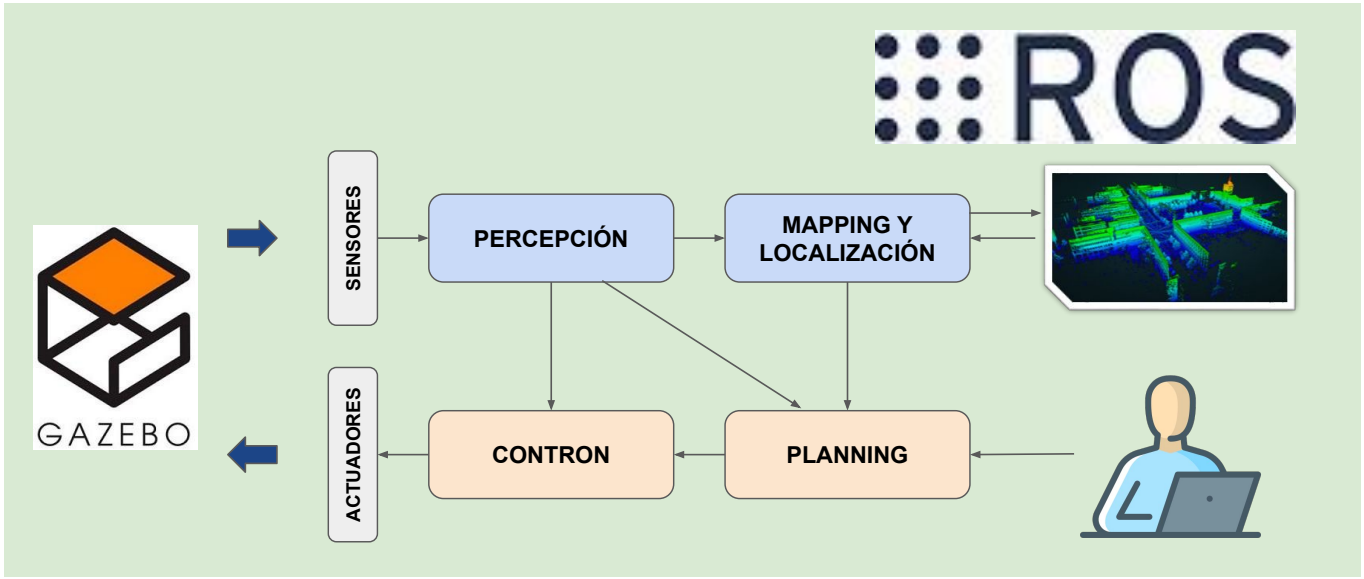
Docker file

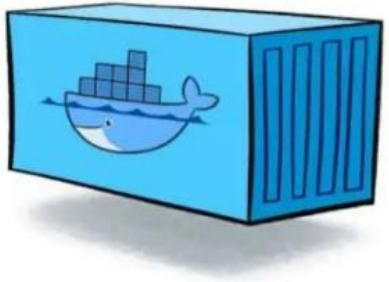
Docker Image

4. Ejecutar el contenedor



Organización software de la **Práctica I**





Docker Container

ROBÓTICA INTELIGENTE

Grado en Ingeniería en Inteligencia Artificial

Introducción a las clases prácticas. Curso 2026-2027

UA
UNIVERSITAT D'ALACANT
UNIVERSIDAD DE ALICANTE
Escola Politècnica Superior
Escuela Politécnica Superior

DFESTS